

Vectors, Embeddings, & Vector Databases

Saraansh Wadkar

1 What Is a Vector?

In the context of embeddings and vector databases, a vector is an n -dimensional ordered list of numbers:

$$\mathbf{v} = [v_1, v_2, \dots, v_n].$$

In practice, each coordinate is usually stored as a signed 32-bit floating-point value, so a single vector is often represented as an n -dimensional array of 4-byte floats.

An embedding vector is useful because it encodes semantic meaning into geometry. Rather than storing meaning in words or symbols directly, the model maps text, images, or other data into a point in a high-dimensional space. Items with similar meaning are placed closer together, while items with different meanings are placed farther apart.

For example, two sentences about databases may produce vectors that point in similar directions, even if they do not share the exact same words. This is what makes vector search possible: we compare the geometry of embeddings instead of relying only on exact keyword matches.

1.1 How Embeddings Are Created

The key question is how a piece of text becomes a useful vector at all. An embedding model first breaks text into smaller units called tokens. Those tokens are then processed by a neural network that has been trained on very large amounts of data. During training, the model learns internal patterns about which words, phrases, and concepts tend to appear in similar contexts.

By the time the model is used for embeddings, it has learned to map text into a high-dimensional latent space. This means that the output numbers are not chosen by hand and do not have obvious human-readable meanings on their own. More precisely, each dimension can end up carrying some semantic signal, so the coordinates are not meaningless random values. However, those meanings are not directly controlled by us or cleanly labeled ahead of time. They emerge during training and are usually spread across many dimensions in a way that is difficult to interpret exactly. Instead, the full vector matters as a whole. The position of that vector reflects the relationships the model learned during training. For a deeper intuition, 3Blue1Brown has a short video on how directions or dimensions in these spaces can pick up semantic meaning in this 3Blue1Brown video.

For example, suppose we embed the sentence “the cat sat on the mat.” The model first tokenizes it into pieces such as “the”, “cat”, “sat”, “on”, “the”, and “mat”. Those tokens

are passed through many layers of the network, which combine them into a single output vector such as

$$\mathbf{e}_{\text{cat}} = [0.12, -0.44, 0.91, \dots, 0.08].$$

The exact numbers are not human-interpretable on their own. What matters is that another sentence like “a kitten is resting on a rug” will likely produce a vector in a nearby region of the space, because the model has learned that “cat” and “kitten” are related concepts and that both sentences describe similar situations.

As a result, text with similar meaning tends to produce vectors that end up near each other, even when the wording changes. The embedding works because the model has compressed many layers of statistical and semantic structure into a numerical representation that preserves useful similarity relationships.

1.2 Semantic Search Example

Suppose you are shopping for a car and want something good for off-road adventures. You have a small catalog of cars, each with a short description. At first this sounds like a simple search problem, but there is an immediate issue: none of the descriptions may literally contain the phrase “off-road adventures.” A plain keyword search or **Ctrl+F** style lookup would therefore miss relevant cars even if some of them are actually a good fit.

Here is the dataset:

1. “Tesla Model 3 is an electric sedan designed for daily commuting with great efficiency”
2. “Ford F-150 is a powerful pickup truck built for hauling and towing heavy loads”
3. “Toyota RAV4 is a compact SUV with good fuel economy and family-friendly features”
4. “Porsche 911 is a high-performance sports car built for speed and handling”
5. “Honda Civic is a reliable compact car ideal for commuting”
6. “Chevy Bolt is an affordable electric hatchback for city driving”

The Ford F-150 and Toyota RAV4 might both be relevant to an “off-road adventures” style query, but neither description uses those exact words. So how do we search by meaning instead of by exact text?

The key idea is to embed each description into a vector. Conceptually, each record might look like:

```
{
  car_name: "Tesla Model 3",
  description: "Electric sedan for efficient daily commuting",
  vector: [...]
}
```

The important point is that the **vector** is produced from the **description**. Once all the descriptions are embedded, we do the same thing to the user’s query:

“vehicle good for off-road adventures”

In this situation, the Toyota RAV4 and Ford F-150 might rank higher than the Tesla Model 3 or Honda Civic, even though none of the descriptions contain the exact phrase “off-road adventures.” The system works because the embedding model has placed related meanings in nearby parts of the vector space. This is the core promise of semantic search: retrieve by meaning, not just by literal keyword overlap.

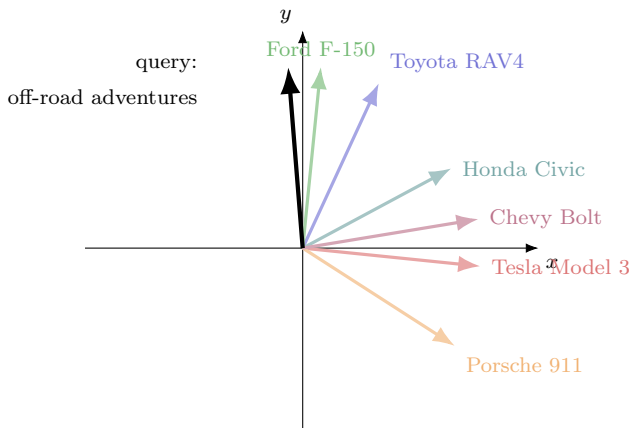


Figure 1: A 2D picture of semantic similarity. Real embeddings are high-dimensional, but the same directional intuition applies.

This picture is only 2D, while real embeddings live in much higher dimensions, but it captures the basic idea. Cars that are close together have similar descriptions. The Tesla Model 3, Honda Civic, and Chevy Bolt cluster in a more commuting- and city-driving-oriented direction. The Toyota RAV4 and Ford F-150 sit much closer to each other because their descriptions both emphasize utility. The query vector for “off-road adventures” also points in that same general direction, which is why those vehicles would be better matches.

At that point, the natural question becomes: how can we mathematically find which vectors are most similar?

1.2.1 Cosine Similarity

Once we have a query vector and a set of car vectors, we need a mathematical way to ask: which vectors point in the most similar direction? One of the most common answers is cosine similarity. Given vectors $\mathbf{u}$ and $\mathbf{v}$, their cosine similarity is

$$\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}.$$

Intuitively, this is about angle. If two vectors point in almost the same direction, then the angle between them is close to 0° , and $\cos(0^\circ) = 1$. That means the vectors are highly similar. If the angle is around 90° , then $\cos(90^\circ) = 0$, which means they are effectively unrelated. If they point in opposite directions, the angle approaches 180° and the cosine becomes negative.

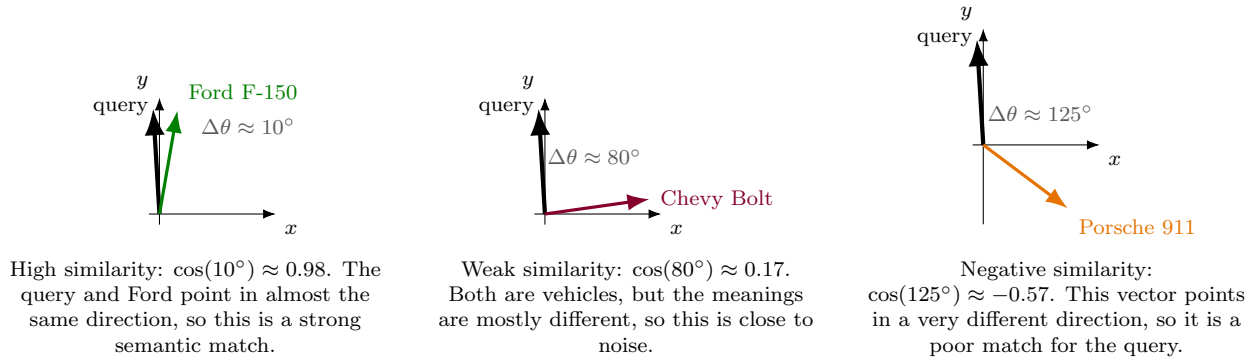


Figure 2: Cosine similarity compares the angle between the query vector and each car vector. Small angles mean high similarity; large angles mean weak or even negative similarity.

We can make that concrete by isolating the query vector and comparing it against a few individual cars:

In the car-shopping example, we would compute the cosine similarity between the query vector and each car description vector, then sort the results from highest to lowest score. If the query vector points in a similar direction to the Toyota RAV4 vector, that suggests their meanings are related even though the query says “off-road adventures” and the description says “compact SUV.” This is exactly why cosine similarity is useful: it lets us retrieve semantically related results without needing the same literal words to appear in both strings.

In practice, cosine similarity scores are often interpreted as rough relevance signals. A score around 0.9 usually indicates a very strong match. Around 0.7 is often still meaningfully related, but less precise. Around 0.4 is more borderline: there may be some overlap in topic or context, but the match is weak. Around 0.2 is often close to noise. These are heuristics rather than hard rules, since the exact thresholds depend on the embedding model and the dataset.

The same idea is not limited to language. Vector representations can also capture similarity for numbers and other structured features. If two data points have similar numerical patterns, their vectors may still end up close together, allowing the database to retrieve values that are numerically or structurally related rather than only textually related.

1.3 Piecing it together

At this point, we have two ingredients: a way to turn data into vectors and a way to compare those vectors. The natural next question is what kind of structure would let us use those ingredients in a real system. Having individual vectors is not enough on its own. We need some organized way to store them alongside the data they came from so that a query can return something useful to a user rather than just a list of floating-point numbers.

The simplest useful structure is a collection of records where each record contains the original content, its embedding vector, and whatever extra metadata we may want to keep around. Conceptually, a single entry might look like this:

```
{
  id: 42,
```

```

    content: "Toyota RAV4 is a compact SUV with good fuel economy",
    embedding: [...],
    metadata: {
      brand: "Toyota",
      type: "SUV"
    }
  }
}

```

Once the data is arranged this way, the workflow becomes much clearer. We embed each document, sentence, image caption, or product description ahead of time and store the resulting vector with the original record. Later, when a user submits a query, we embed the query into its own vector, compare it against the stored vectors, and return the records whose embeddings are closest. In other words, the useful structure is not just “a pile of vectors,” but a searchable collection of data where each vector remains tied to the object it represents.

This is the basic idea that leads directly to vector databases. A typical vector database is built around storing many of these vector-backed records, keeping them associated with IDs and metadata, and making nearest-neighbor lookup fast enough to be practical even when the dataset becomes very large. We will look at how vector databases usually do that in the next section, but for now the important idea is simple: embeddings become useful when they are placed inside a structure that supports retrieval.